

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

ANNEXURE A

INTEGRATED EXPERIMENTS

Code of Conduct:

I NARAPPAGARI SRAVYA(192472192)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

ANNEXURE A

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).
- (b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- (c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

ANNEXURE A

EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective

To implement and evaluate a **Decision Tree Classifier** using the **Iris dataset** in Python.

(a) Load and Pre-process the Dataset

Python Code

```
# Import required libraries

import pandas as pd

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier, export_text

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load Iris dataset

iris = load_iris()

# Create DataFrame

df = pd.DataFrame(iris.data, columns=iris.feature_names)

df["target"] = iris.target

# Display first 5 rows

print("First 5 Rows:")

print(df.head())

# Display data types

print("\nData Types:")

print(df.dtypes)

# Check missing values

print("\nMissing Values:")

print(df.isnull().sum())
```

```
# Separate input and output

X = df.drop("target", axis=1)

y = df["target"]

# Split into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(

    X, y,

    test_size=0.2,

    random_state=42

)
```

Expected First 5 Rows

sepal length	sepal width	petal length	petal width	target
5.1	3.5	1.4	0.2	0
4.9	3.0	1.4	0.2	0
4.7	3.2	1.3	0.2	0
4.6	3.1	1.5	0.2	0
5.0	3.6	1.4	0.2	0

Data Types

sepal length (cm) float64

sepal width (cm) float64

petal length (cm) float64

petal width (cm) float64

target int64

Pre-processing

For the Iris dataset:

- There are no missing values.
- All input features are numerical.
- Therefore, encoding is not required.

- The dataset is split into:
 - **80% Training Data**
 - **20% Testing Data**

(b) Build the Decision Tree Model

Using Gini Index

Create Decision Tree model

```
model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)
```

Train the model

```
model.fit(X_train, y_train)
```

Print tree structure

```
tree_rules = export_text(
    model,
    feature_names=list(X.columns)
)

print(tree_rules)
```

Root Node

The root node can be identified using:

```
print("Root Feature:", X.columns[model.tree_.feature[0]])
```

The root node is the feature selected for the **first split** by the trained model.

Justification

The Decision Tree selects the root feature because it provides the **best split according to the selected criterion**.

For the **Gini Index**, the algorithm selects the feature that produces the **maximum reduction in impurity**.

X₁

X₁

X2

X2

Give feedback

(c) Evaluate the Model

Python Code

```
# Make predictions
```

```
y_pred = model.predict(X_test)
```

```
# Accuracy
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
```

```
# Confusion Matrix
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
print("\nConfusion Matrix:")
```

```
print(cm
```

```
# Classification Report
```

```
print("\nClassification Report:")
```

```
print(classification_report(
```

```
    y_test,
```

```
    y_pred,
```

```
    target_names=iris.target_names
```

```
))
```

Find Misclassified Instances

```
# Find incorrect predictions
```

```
misclassified = df.loc[
```

```
    X_test.index[y_test != y_pred]
```

```
]
```

```
print("\nMisclassified Instances:")
```

```
print(misclassified)
```

Alternative Method

```
results = X_test.copy()
```

```
results["Actual"] = y_test
```

```
results["Predicted"] = y_pred
```

```
misclassified = results[
```

```
    results["Actual"] != results["Predicted"]
```

```
]
```

```
print(misclassified.head(2))
```

Performance Comment

The Decision Tree usually performs very well on the Iris dataset because the dataset has clear patterns between flower classes.

Performance observations:

- High classification accuracy
- Easy to understand
- No feature scaling required
- Can classify nonlinear relationships
- May overfit if the tree becomes too deep

Important Note About Misclassified Instances

The **exact accuracy, confusion matrix, root node, and misclassified rows depend on the train-test split and the trained tree**. With `random_state=42`, your Python output will provide the exact results for your experiment.

Experiment 2: Reinforcement Learning – Q-Learning

Objective

To implement **Q-Learning** in a **4 × 4 Grid World** and observe how an agent learns the optimal policy.

(a) Define the Environment

Grid World

S . . .

. X . .

. . X .

. . . G

Where:

- **S** = Starting position
- **G** = Goal
- **X** = Obstacle
- **.** = Normal state

Actions

The agent can move:

- Up
- Down
- Left
- Right

Rewards

Condition	Reward
Reach Goal	+10
Normal Step	-1
Obstacle	-5

Hyperparameters

We use:

Learning Rate (α) = 0.1

Discount Factor (γ) = 0.9

Exploration Rate (ϵ) = 0.1

Episodes = 100

Python Code – Environment and Q-Table

```
import numpy as np
```

```
import random
```

```
import matplotlib.pyplot as plt
```

```
# Grid size
```

```
GRID_SIZE = 4
```

```
# States and actions
```

```
ACTIONS = ["Up", "Down", "Left", "Right"]
```

```
# Action indexes
```

```
UP = 0
```

```
DOWN = 1
```

```
LEFT = 2
```

```
RIGHT = 3
```

```
# Start and goal
```

```
START = (0, 0)
```

```
GOAL = (3, 3)
```

```
# Obstacles
```

```
OBSTACLES = [(1, 1), (2, 2)]
```

```
# Hyperparameters
```

```
alpha = 0.1
```

```
gamma = 0.9
```

```
epsilon = 0.1

# Number of states

n_states = GRID_SIZE * GRID_SIZE

# Initialize Q-table with zeros

Q = np.zeros((n_states, 4))

# Convert position to state number

def state_index(position):

    return position[0] * GRID_SIZE + position[1]
```

Movement Function

```
def move(state, action):

    row, col = state

    if action == UP:

        row = max(0, row - 1)

    elif action == DOWN:

        row = min(GRID_SIZE - 1, row + 1)

    elif action == LEFT:

        col = max(0, col - 1)

    elif action == RIGHT:

        col = min(GRID_SIZE - 1, col + 1)

    new_state = (row, col)

    # Obstacle penalty

    if new_state in OBSTACLES:

        reward = -5

        new_state = state

    # Goal reward

    elif new_state == GOAL:

        reward = 10
```

```
# Normal movement
```

```
else:
```

```
    reward = -1
```

```
    return new_state, reward
```

(b) Run Q-Learning for 100 Episodes

Q-Learning Algorithm

The agent updates its Q-value using:

$$Q(s, a) = Q(s, a) + \alpha [R + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Python Code

```
episodes = 100
```

```
# Store cumulative rewards
```

```
episode_rewards = []
```

```
# Store Q-values for representative states
```

```
q_history = {}
```

```
# Representative states
```

```
representative_states = [
```

```
    (0, 0),
```

```
    (1, 0)
```

```
]
```

```
for episode in range(1, episodes + 1):
```

```
    state = START
```

```
    total_reward = 0
```

```
    done = False
```

```
    steps = 0
```

```
    max_steps = 100
```

```

while not done and steps < max_steps:

    s = state_index(state)

    # Epsilon-greedy action selection
    if random.uniform(0, 1) < epsilon:

        action = random.randint(0, 3)

    else:

        action = np.argmax(Q[s])

    # Perform action
    next_state, reward = move(state, action)

    ns = state_index(next_state)

    # Q-learning update
    Q[s, action] = Q[s, action] + alpha * (

        reward +

        gamma * np.max(Q[ns]) -

        Q[s, action]

    )

    total_reward += reward

    state = next_state

    steps += 1

    # Check if goal is reached
    if state == GOAL:

        done = True

episode_rewards.append(total_reward)

# Record Q-values
if episode in [1, 50, 100]:

    q_history[episode] = {}

```

```

for pos in representative_states:

    idx = state_index(pos)

    q_history[episode][pos] = Q[idx].copy()

# Display recorded Q-values

for episode, values in q_history.items():

    print(f"\nEpisode {episode}")

    for state, q_values in values.items():

        print(

            "State:", state,

            "Q-values:", q_values

        )

```

Example of Q-Table Evolution

The exact values can change because Q-Learning includes random exploration.

Episode 1

Initially:

State (0,0): [0.0, 0.0, 0.0, 0.0]

State (1,0): [0.0, 0.0, 0.0, 0.0]

The agent has little or no knowledge of the environment.

Episode 50

Q-values begin to change.

The agent starts learning:

- Which actions lead toward the goal
- Which actions should avoid obstacles
- Which movements give better rewards

Episode 100

The best actions usually have higher Q-values.

The agent has learned a better path toward the goal.

Note: The exact Q-values at Episodes 1, 50, and 100 depend on random exploration.

(c) Extract the Final Learned Policy

Python Code

```
# Symbols for actions
action_symbols = {
    0: "↑",
    1: "↓",
    2: "←",
    3: "→"
}

print("\nFinal Learned Policy:\n")

for row in range(GRID_SIZE):
    policy_row = []
    for col in range(GRID_SIZE):
        position = (row, col)
        if position == GOAL:
            policy_row.append("G")
        elif position in OBSTACLES:
            policy_row.append("X")
        else:
            state = state_index(position)
            best_action = np.argmax(Q[state])
            policy_row.append(
                action_symbols[best_action]
            )
    print(policy_row)
```

Expected Policy Format

A learned policy may look similar to:

↓ → → ↓

↓ X → ↓

↓ ↓ X ↓

→ → → G

Note: The exact policy can vary depending on random exploration and training.

Plot Cumulative Reward per Episode

```
plt.plot(
    range(1, episodes + 1),
    episode_rewards
)

plt.xlabel("Episode")

plt.ylabel("Cumulative Reward")

plt.title("Q-Learning: Cumulative Reward per Episode")

plt.grid()

plt.show()
```

Convergence Behaviour

Initial Episodes

At the beginning:

- Q-values are zero.
- The agent explores randomly.
- The agent may hit obstacles.
- Total reward is usually low.

Middle Episodes

As learning continues:

- Q-values improve.

- The agent starts avoiding obstacles.
- Better paths are discovered.
- Rewards generally improve.

Final Episodes

After sufficient learning:

- The agent prefers actions with high Q-values.
- The learned policy becomes more stable.
- The agent reaches the goal using a better path.
- The cumulative reward generally improves.

Final Conclusion

The Q-Learning agent successfully learns from interaction with the 4×4 **Grid World**. Initially, the agent explores randomly, but after repeated episodes, the **Q-table stores useful knowledge about actions and rewards**. The final policy selects the best action for each state and guides the agent toward the goal while avoiding obstacles.